Q1. Introduce Yourself

This is a common opening question in interviews. You should focus on summarizing your background, education, and key skills concisely.

"Hello, my name is [Your Name]. I graduated with a degree in [Your Degree] from [Your University], where I gained a strong foundation in [mention key technical skills or areas of interest, like programming languages, software development, data structures, or web development]. Over the last [X] years, I have worked on several projects involving [mention specific technologies, such as Python, Java, C#, or web frameworks like Angular or ASP.NET]. I also have experience in [mention any key areas relevant to the role, such as algorithms, database management, or software design]. In addition to technical skills, I am passionate about problem-solving, continuously learning, and applying my knowledge to real-world challenges."

Q2. What projects have you worked on?

In this question, the interviewer is looking for examples that highlight your problem-solving skills, technical knowledge, and project experience.

"I've worked on several projects throughout my academic career and personal time. One notable project was [mention a specific project, like developing a web application, building an algorithm, or creating a software tool]. In this project, I was responsible for [briefly explain your role, e.g., designing the user interface, implementing backend functionality, or optimizing performance]. I utilized [mention the technologies used, such as Python, JavaScript, C++, databases, or frameworks]. One of the key challenges I encountered was [mention a challenge], but I was able to [explain how you solved the problem]."

Q3. Explain the concept of a binary tree and its traversal algorithms

This is a technical question testing your understanding of data structures. Be clear and concise in your explanation.

"A binary tree is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. It's commonly used in searching and sorting

algorithms. There are three primary ways to traverse a binary tree:

- **1. In-order Traversal:** First, we visit the left subtree, then the root node, and finally the right subtree. This method is often used for binary search trees because it retrieves nodes in non-decreasing order.

- **2. Pre-order Traversal:** We visit the root node first, then recursively visit the left subtree followed by the right subtree. This traversal is useful for creating a copy of the tree or prefix notation of an expression.
- **3. Post-order Traversal:** In this method, we recursively visit the left subtree, then the right subtree, and finally the root node. Post-order traversal is commonly used to delete or free nodes in a tree, as it ensures children are processed before the parent node."

Technical Questions

These questions focus on assessing your proficiency in programming and computer science concepts.

Expect questions on:

- Data structures (linked lists, stacks, queues)
- Algorithms (sorting, searching)
- Object-oriented programming concepts
- Database queries (SQL)

Q4. What is the difference between a linked list and an array?

Answer:

- **Array:** Fixed size, stored in contiguous memory locations, and supports random access.
- **Linked List:** Dynamic size, stored in non-contiguous memory locations, and requires sequential access.

Q5. How would you reverse a linked list?

Answer:

1. Initialize three pointers: `prev = NULL`, `current = head` and `next = NULL`.
2. Traverse the list:
    - Store `current->next` in `next`.
    - Set `current->next` to `prev`.
    - Move `prev` and `current` one step ahead.
3. Update the head to `prev`.

Example

```
Node* reverse(Node* head) {
   Node* prev = NULL;
```

```
    Node* current = head;
    Node* next = NULL;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }

    return prev;
}
```

Q6. How does a stack differ from a queue?

Answer:

- **Stack:** Follows LIFO (Last In, First Out). Operations: `push`, `pop`.
- **Queue:** Follows FIFO (First In, First Out). Operations: `enqueue`, `dequeue`.

Q7. Write a program to check for balanced parentheses using a stack.

Answer:

Logic: Push opening brackets onto the stack and pop when a closing bracket is encountered. The stack should be empty at the end if balanced.

```
def is_balanced(expression):
    stack = []
    for char in expression:
        if char in '([{':
            stack.append(char)
        elif char in ')]}':
            if not stack:
                return False
            top = stack.pop()
            if (char == ')' and top != '(') or \
               (char == ']' and top != '[') or \
               (char == '}' and top != '{'):
                return False
    return not stack
```

Q8. Explain the concept of OOPs and its four pillars.

Answer:

- **Encapsulation:** Wrapping data and methods into a single unit (class).
- **Abstraction:** Hiding implementation details and showing only functionality.
- **Inheritance:** Acquiring properties of a parent class in a child class.
- **Polymorphism:** Performing a single action in different ways (e.g., method overloading, overriding).

Q9. What is the difference between method overloading and method overriding?

Answer:

- **Method Overloading:** Same method name, different parameters, happens at compile-time.
- **Method Overriding:** Same method name, same parameters in parent and child classes, happens at runtime.

Q10. Write a SQL query to find the second-highest salary from the Employee table.

Answer:

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Q11. What is the difference between DELETE and TRUNCATE in SQL?

Answer:

- **DELETE:** Removes specific rows, can use a `WHERE` clause, logs each row deletion, and is slower.
- **TRUNCATE:** Removes all rows, cannot use a `WHERE` clause, does not log individual deletions in the same way, and is faster.

Q12. Explain Bubble Sort and its time complexity.

Answer:

Bubble Sort: Repeatedly compare adjacent elements and swap them if they are in the wrong order.

Time Complexity:

- **Best Case:** `O(n)` (when the array is already sorted).
- **Worst Case:** `O(n^2)`.

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Q13. Write a query to fetch all employees earning more than the average salary.

Answer:

```
SELECT *
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Q18. Explain the difference between JDK, JRE, and JVM in Java.

Answer:

- **JDK (Java Development Kit):** A development environment that includes tools like a compiler, debugger, and libraries needed to develop Java applications.
- **JRE (Java Runtime Environment):** A runtime environment that provides the libraries and JVM required to run Java applications.
- **JVM (Java Virtual Machine):** An engine that executes Java bytecode. It provides platform independence by converting bytecode into machine code.

Q19. Write a Python program to find the longest palindrome in a string.

Answer:

```
def longest_palindrome(s):
    def expand_around_center(left, right):
```

```
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left + 1:right]

    result = ""
    for i in range(len(s)):
        odd_palindrome = expand_around_center(i, i)
        even_palindrome = expand_around_center(i, i + 1)
        result = max(result, odd_palindrome, even_palindrome, key=len)

    return result

# Example usage
print(longest_palindrome("babad"))  # Output: "bab" or "aba"
```

Q21. Explain the TCP/IP model and its layers.

Answer:

- **Application Layer:** Provides user services like email (SMTP), file transfer (FTP), and web browsing (HTTP).
- **Transport Layer:** Ensures reliable data transfer using protocols like TCP (reliable, connection-oriented) and UDP (unreliable, connectionless).
- **Internet Layer:** Handles packet routing through logical addressing (IP addresses) using protocols like IPv4 and IPv6.
- **Network Interface Layer:** Manages physical connections and data transmission through protocols like Ethernet and Wi-Fi.

Q22. Explain the concept of a binary search tree (BST) and write its insertion algorithm.

Answer:

- **Binary Search Tree (BST):** A tree data structure where each node has at most two children. The left child contains values less than the parent node, and the right child contains values greater than the parent node.

Insertion Algorithm

```
class Node:

    def __init__(self, key):

        self.key = key

        self.left = None

        self.right = None
```

```
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
root = insert(root, 70)[cite: 3]
print("Node inserted successfully")[cite: 3]
```

Q27. What are the different layers of the OSI model?

The OSI (Open Systems Interconnection) model consists of seven layers that define how data is transmitted and received over a network.

These layers are:

1. **Physical Layer:** Handles the transmission of raw bit streams over physical mediums like cables and switches.
2. **Data Link Layer:** Ensures error-free data transfer between adjacent nodes using protocols like Ethernet.
3. **Network Layer:** Manages data routing, forwarding, and logical addressing (e.g., IP addresses).
4. **Transport Layer:** Provides end-to-end communication, ensuring reliable data transfer (e.g., TCP/UDP).
5. **Session Layer:** Manages and controls connections between devices, including session creation and termination.
6. **Presentation Layer:** Translates data between application and network formats, ensuring proper encoding, encryption, and compression.
7. **Application Layer:** Interfaces with the user, providing services like email (SMTP), file transfer (FTP), and web browsing (HTTP). This model helps standardize networking processes and facilitates interoperability between devices and technologies.